

Week 2B Classification_Metrics

AUTHOR
Andre Thomson

Approach

I used the penguin predictions file from the course materials. I first checked the number of female and male penguins and calculated the null error rate. I treated female as the positive class.

Next, I used the predicted probability of female to test thresholds of 0.2, 0.5, and 0.8. For each threshold, I counted the true positives, false positives, true negatives, and false negatives. I then calculated accuracy, precision, recall, and F1 score and compared the results.

Data

The file has 93 observations. The predicted class column uses the words female and male, although the assignment description refers to 1 and 0. I did not use that column for the threshold calculations. I made the new predictions from the probability of female.

I used R and Quarto for the analysis. I checked the code and file paths in Visual Studio Code before rendering the report.

Load packages and data

Source: https://github.com/acatlin/data/blob/master/penguin_predictions.csv

```
library(tidyverse)
library(knitr)
library(scales)

penguins <- read_csv(
  "penguin_predictions.csv",
  show_col_types = FALSE
)

required_columns <- c(".pred_female", ".pred_class", "sex")
stopifnot(all(required_columns %in% names(penguins)))
stopifnot(nrow(penguins) == 93)
stopifnot(all(penguins$.pred_female >= 0 & penguins$.pred_female <= 1))
stopifnot(all(str_to_lower(penguins$sex) %in% c("female", "male"))))

glimpse(penguins)
```

Rows: 93

Columns: 3

\$.pred_female <dbl> 0.99217462, 0.95423945, 0.98473504, 0.18702056, 0.9947012...

\$.pred_class <chr> "female", "female", "female", "male", "female", "female",...

\$ sex <chr> "female", "female", "female", "female", "female", "female",...

Prepare the actual class

Female is the positive class and male is the negative class.

```
penguins <- penguins %>%
  mutate(
    actual = if_else(
      str_to_lower(sex) == "female",
      1L,
      0L
    )
  )

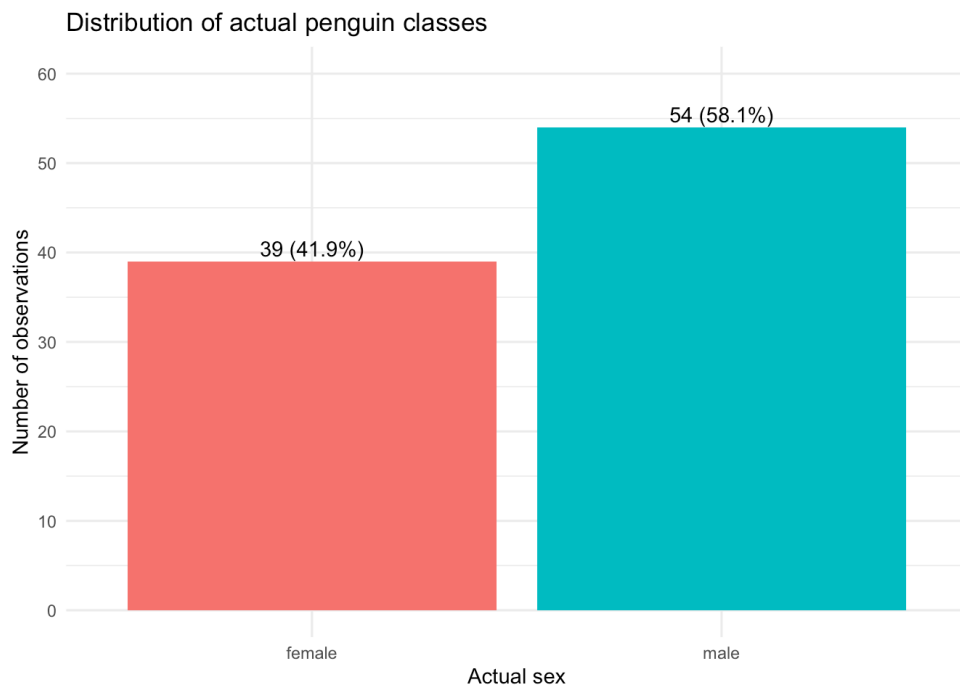
class_distribution <- penguins %>%
  count(sex) %>%
  mutate(percent = n / sum(n))
```

```
kable(class_distribution)
```

sex	n	percent
female	39	0.4193548
male	54	0.5806452

Plot the actual class distribution

```
ggplot(  
  class_distribution,  
  aes(x = sex, y = n, fill = sex)  
) +  
  geom_col(show.legend = FALSE) +  
  geom_text(  
    aes(label = paste0(n, " (", scales::percent(percent, accuracy = 0.1), "%)"),  
    vjust = -0.35  
  ) +  
  scale_y_continuous(  
    limits = c(0, 60),  
    breaks = seq(0, 60, 10)  
  ) +  
  labs(  
    title = "Distribution of actual penguin classes",  
    x = "Actual sex",  
    y = "Number of observations"  
  ) +  
  theme_minimal()
```



There are 39 female penguins and 54 male penguins. Male is the majority class.

Calculate the null error rate

```
majority_accuracy <- max(class_distribution$n) / sum(class_distribution$n)  
null_error_rate <- 1 - majority_accuracy  
  
null_error_result <- tibble(  
  majority_class = class_distribution$sex[which.max(class_distribution$n)],
```

```
majority_accuracy = majority_accuracy,
null_error_rate = null_error_rate
)

kable(null_error_result, digits = 4)
```

majority_class	majority_accuracy	null_error_rate
male	0.5806	0.4194

The null error rate is 0.4194, or 41.94%. This is the error from always predicting the majority class. I used it as the baseline for comparing the model.

Calculate the confusion matrices and metrics

```
calculate_metrics <- function(data, threshold) {
  evaluated <- data %>%
    mutate(
      predicted = if_else(.pred_female > threshold, 1L, 0L)
    )

  tp <- sum(evaluated$predicted == 1 & evaluated$actual == 1)
  fp <- sum(evaluated$predicted == 1 & evaluated$actual == 0)
  tn <- sum(evaluated$predicted == 0 & evaluated$actual == 0)
  fn <- sum(evaluated$predicted == 0 & evaluated$actual == 1)

  accuracy <- (tp + tn) / (tp + fp + tn + fn)
  precision <- if_else(tp + fp == 0, NA_real_, tp / (tp + fp))
  recall <- if_else(tp + fn == 0, NA_real_, tp / (tp + fn))
  f1 <- if_else(
    is.na(precision) | is.na(recall) | precision + recall == 0,
    NA_real_,
    2 * precision * recall / (precision + recall)
  )

  tibble(
    threshold = threshold,
    TP = tp,
    FP = fp,
    TN = tn,
    FN = fn,
    accuracy = accuracy,
    precision = precision,
    recall = recall,
    F1 = f1
  )
}

threshold_results <- map_dfr(
  c(0.2, 0.5, 0.8),
  ~ calculate_metrics(penguins, .x)
)

expected_counts <- tribble(
  ~threshold, ~TP, ~FP, ~TN, ~FN,
  0.2, 37, 6, 48, 2,
  0.5, 36, 3, 51, 3,
  0.8, 36, 2, 52, 3
)

stopifnot(all(threshold_results$TP + threshold_results$FP +
  threshold_results$TN + threshold_results$FN == nrow(penguins)))
stopifnot(
  isTRUE(
    all.equal(
      threshold_results %>% select(threshold, TP, FP, TN, FN),
      expected_counts,
      check.attributes = FALSE
    )
  )
)
```

```

    )
  )
)

kable(
  threshold_results,
  digits = 3
)

```

threshold	TP	FP	TN	FN	accuracy	precision	recall	F1
0.2	37	6	48	2	0.914	0.860	0.949	0.902
0.5	36	3	51	3	0.935	0.923	0.923	0.923
0.8	36	2	52	3	0.946	0.947	0.923	0.935

Confusion matrices

The rows show the actual class and the columns show the predicted class.

```

create_confusion_matrix <- function(data, threshold) {
  data %>%
    mutate(
      actual_class = if_else(actual == 1L, "Female", "Male"),
      predicted_class = if_else(
        .pred_female > threshold,
        "Female",
        "Male"
      )
    ) %>%
    count(actual_class, predicted_class) %>%
    complete(
      actual_class = c("Female", "Male"),
      predicted_class = c("Female", "Male"),
      fill = list(n = 0)
    ) %>%
    pivot_wider(
      names_from = predicted_class,
      values_from = n
    )
}

for (threshold in c(0.2, 0.5, 0.8)) {
  cat("\n\n### Threshold", threshold, "\n\n")
  print(kable(create_confusion_matrix(penguins, threshold)))
}

```

Threshold 0.2

actual_class	Female	Male
Female	37	2
Male	6	48

Threshold 0.5

actual_class	Female	Male
Female	36	3
Male	3	51

Threshold 0.8

actual_class	Female	Male
Female	36	3
Male	2	52

Compare the thresholds

At 0.2, recall is 0.949, which is the highest recall of the three thresholds. It finds 37 of the 39 female penguins, but it also gives 6 false positives.

At 0.8, accuracy is 0.946, precision is 0.947, and F1 is 0.935. Moving from 0.5 to 0.8 removes one false positive and does not add another false negative in this dataset.

The 0.8 threshold gives the best accuracy and F1 for this dataset. That does not make it the best threshold for every situation. It depends on which mistake matters more.

Threshold use cases

A threshold of 0.2 could be used for an initial security scan of school computers. It would flag more devices for review. That can create more false alarms, but a technician can check the devices before taking action. In this case, missing an infected device may be worse than checking a safe one.

A threshold of 0.8 could be used before automatically disabling a staff account. It would require stronger evidence before blocking access. This can reduce unnecessary lockouts, but some suspicious accounts may not be stopped right away.

Conclusion

The threshold changed the number of false positives and false negatives. The 0.2 threshold found more female cases but produced more false positives. The 0.8 threshold had the highest accuracy, precision, and F1 for this dataset. The best threshold depends on which error creates the bigger problem.

References

- Catlin, A. *Performance Metrics for Classification Problems in Machine Learning*. Course GitHub repository. <https://github.com/acatlin/data/blob/master/Performance%20Metrics%20for%20Classification%20problems%20in%20Machine%20Learning.pdf>
- Catlin, A. Penguin predictions dataset. Course GitHub repository. https://github.com/acatlin/data/blob/master/penguin_predictions.csv
- Posit. Read CSV documentation. https://readr.tidyverse.org/reference/read_delim.html
- Posit. Bar chart documentation. https://ggplot2.tidyverse.org/reference/geom_bar.html
- Google for Developers. *Thresholds and the Confusion Matrix*. <https://developers.google.com/machine-learning/crash-course/classification/thresholding>
- Google for Developers. *Accuracy, Recall, Precision, and Related Metrics*. <https://developers.google.com/machine-learning/crash-course/classification/accuracy-precision-recall>
- StatQuest with Josh Starmer. *Machine Learning Fundamentals: The Confusion Matrix*. YouTube. <https://www.youtube.com/watch?v=Kdsp6soqA7o>